

# Fractal Dimension Computation Methods for Audio Signals

Fractal dimension (FD) analysis provides a quantitative measure of the complexity or “roughness” of a signal’s waveform. In audio processing, fractal dimensions can capture subtle differences in signal characteristics that may help distinguish genuine speech from AI-generated (deepfake) speech. Below, we detail several FD computation methods applicable to audio waveforms – including Multifractal Detrended Fluctuation Analysis (MFDFA), Higuchi’s method, Katz’s method, Petrosian’s method, and the Box-Counting algorithm – with their algorithms, preprocessing needs, global/local usage, and relevance to spoofed vs. genuine speech detection.

## Multifractal Detrended Fluctuation Analysis (MFDFA)

*MFDFA* is a robust technique to estimate **fractal scaling exponents across multiple scales** (yielding a multifractal spectrum) for a time series <sup>1</sup> <sup>2</sup>. It generalizes the classic Detrended Fluctuation Analysis (DFA) to capture both monofractal and multifractal properties. MFDFA is well-suited to nonstationary signals like audio, as it removes polynomial trends and examines fluctuations at various window sizes <sup>3</sup> <sup>4</sup>. This provides a **global multifractal characterization** of the audio (though it can be applied on sliding windows for local analysis if needed). The multifractal spectrum can reveal differences in the distribution of small vs. large fluctuations, which might differ between natural and deepfake speech.

**Algorithm (MFDFA)** – Given an audio waveform  $x[n]$  of length  $N$  (after converting to a 1D signal, e.g. a mono WAV):

1. **Integration (“Profile” construction):** Convert the signal into a random-walk-like profile  $Y(i)$  by cumulative summation <sup>5</sup>. Typically subtract the mean first to avoid a linear trend. For  $i = 1, \dots, N$ :

$$Y(i) = \sum_{k=1}^i [x_k - \langle x \rangle],$$

where  $\langle x \rangle$  is the mean amplitude <sup>6</sup>. This step integrates the “noise-like” audio into a nonstationary “walk” signal suitable for DFA <sup>5</sup>.

2. **Segmentation into windows:** Choose a set of window lengths (scales)  $s$  (e.g. spanning samples from short to long durations). Divide the profile  $Y(i)$  into non-overlapping segments of length  $s$ . Since  $N$  may not be a multiple of  $s$ , perform this from the start and end, yielding  $2N_s$  segments (where  $N_s = \lfloor N/s \rfloor$ ) <sup>7</sup>.
3. **Local detrending and fluctuation:** For each segment  $\nu = 1, \dots, 2N_s$ , fit a polynomial (of order  $m$ ) to the profile in that segment (this models the local trend) <sup>3</sup>. Compute the variance (fluctuation) around the trend:

$$F^2(\nu, s) = \frac{1}{s} \sum_{i=1}^s \left( Y[(\nu - 1)s + i] - y_\nu(i) \right)^2,$$

where  $y_\nu(i)$  is the fitted polynomial in segment  $\nu$  <sup>8</sup> <sup>9</sup>. This detrending removes low-frequency trends (of order  $m - 1$  in the original signal, if using an order- $m$  fit) so that only intrinsic fluctuations remain <sup>10</sup>. In practice, linear detrending (DFA-1) or quadratic (DFA-2) is often sufficient for audio.

4. **Compute q-order fluctuation function:** Aggregate the fluctuations from all segments to characterize scale-dependent behavior. For any real exponent  $q$ , define:

$$F_q(s) = \left\{ \frac{1}{2N_s} \sum_{\nu=1}^{2N_s} \left[ F^2(\nu, s) \right]^{q/2} \right\}^{1/q},$$

for  $q \neq 0$  <sup>11</sup>. (For  $q = 0$ , use the geometric mean  $F_0(s) \equiv \exp\{\frac{1}{2N_s} \sum_{\nu} \ln F^2(\nu, s)\}$  <sup>12</sup>.) Varying  $q$  emphasizes different fluctuation magnitudes: *large* fluctuations dominate positive  $q$ , while *small* fluctuations influence negative  $q$ . Note that  $q = 2$  reproduces the standard DFA (monofractal) fluctuation function <sup>13</sup>.

5. **Determine scaling behavior:** Repeat Steps 2–4 for multiple window sizes  $s$ . Plot  $\log F_q(s)$  vs.  $\log s$  for each  $q$ . In the regime where a power-law holds, the slope gives the *generalized Hurst exponent*  $h(q)$  <sup>14</sup>:

$$F_q(s) \sim s^{h(q)}.$$

For a **monofractal** time series,  $h(q)$  is constant (independent of  $q$ ) – essentially the single Hurst exponent  $H = h(2)$  describing the global roughness <sup>15</sup>. A **multifractal** series yields a decreasing  $h(q)$  for higher  $q$  (indicating that large fluctuations scale differently than small ones).

6. **Compute fractal spectrum (if needed):** From  $h(q)$ , one can obtain the multifractal spectrum  $D(q)$  or  $f(\alpha)$  via the Legendre transform. For example, the classical partition function formalism relates  $h(q)$  to the mass exponent  $\tau(q)$  as  $\tau(q) = q h(q) - 1$  for stationary signals <sup>16</sup>. The singularity spectrum  $f(\alpha)$  is then given by  $f(\alpha) = q \alpha - \tau(q)$ , where  $\alpha = h(q)$  is the Hölder exponent. In practice, one can derive the spectrum  $f(\alpha)$  after estimating  $h(q)$  across the range of  $q$ .

**Preprocessing:** MFDFA inherently handles trends and nonstationarity by detrending in each window. Thus, minimal preprocessing is required beyond converting the audio to a numeric time-series. It is often beneficial to **normalize the audio amplitude** (e.g. zero-mean as done above, and perhaps unit variance) so that results are comparable across files. Extremely low-energy sections (silence) may be excluded or analyzed separately, since silence contributes very low fluctuations which could skew the multifractal spectrum. If the audio contains a lot of high-frequency noise, a mild smoothing or denoising could be applied – however, be cautious, as removing genuine fine-scale fluctuations might also remove information relevant to fractal analysis. Typically, one uses the raw waveform or a band-limited waveform (e.g. 16 kHz sampling focused on speech band) for computing MFDFA features.

**Global vs. local:** MFDFA is primarily used to get a **global multifractal signature** of the entire signal (or a long segment). The output is often a set of scaling exponents or a spectrum describing the **overall** signal complexity. However, it is feasible to perform MFDFA on consecutive frames or sliding windows to obtain a time-varying measure (for instance, one could compute the **monofractal** DFA exponent in a

moving window to track how fractal dimension changes over time). In fact, using a **sliding window DFA** is a known approach in biomedical signals to capture transient changes <sup>17</sup> <sup>18</sup> . For spoof detection, one might compute the **whole-utterance** multifractal spectrum as a feature vector, or compute DFA/Hurst exponents on short frames to feed as a sequence to a classifier. Generally, longer data (several seconds) are needed for a reliable multifractal spectrum; for very short utterances, a single global exponent (like the Hurst  $H = h(2)$  ) might be more stable.

**Relevance to real vs. deepfake speech:** MFDFA can capture subtle differences in the **statistical structure** of audio that may not be evident in conventional spectral features. Genuine human speech signals often exhibit **scale-invariant fluctuations** due to physiological mechanisms (vocal fold micro-tremor, breathing patterns, emotional prosody variations, etc.), potentially yielding a rich multifractal spectrum. Deepfake or synthetic speech might be generated by models that **oversmooth** the signal or impose artificial constraints, potentially leading to a different multifractal signature. For example, a synthetic voice might lack the small-scale variability present in natural speech, resulting in a narrower multifractal spectrum (with certain  $h(q)$  values or spectrum width deviating from a real voice's pattern). MFDFA could thus help distinguish fake from real by providing features such as the spectrum **width** (difference between maximal and minimal  $\alpha$ ) or the shape of  $f(\alpha)$ . These features could indicate, for instance, reduced complexity in a deepfake's amplitude fluctuations. Although MFDFA is computationally heavier than single-dimension methods, its ability to characterize *multiple* scaling behaviors might catch anomalies in deepfake audio that single FD values miss. In summary, MFDFA yields a **multidimensional fractal feature set** that can serve as a fingerprint of an audio signal's complexity.

**Implementation in a pipeline:** To use MFDFA in a batch-processing pipeline (e.g. in Python), one should:

- **Resample** or ensure all audio signals have a consistent sampling rate (e.g. 16 kHz, a common rate for speech <sup>19</sup> ) so that time scales are comparable across samples.
- Optionally perform **voice activity detection (VAD)** to isolate voiced regions; one could compute separate multifractal spectra for voiced segments vs. unvoiced, or simply focus on regions of interest. This avoids dominant silent parts skewing the analysis.
- Choose a range of scales  $s$  appropriate for the sampling rate and signal length. For speech at 16 kHz, scales might range from, say, 20–50 samples ( $\approx$  few milliseconds, capturing fine fluctuations) up to perhaps 0.5–1 second (capturing prosodic variations). Ensure at least  $\sim 4$  segments for the largest scale to fit a line reliably <sup>16</sup> (often one excludes scales above  $N/4$  for stability).
- Use an existing implementation or write one: one can implement the above steps using NumPy for efficiency. In practice, third-party libraries (e.g. `no1ds`) or custom functions can compute DFA, but for multifractal one may have to code the loop over  $q$  and scales. The **polynomial detrending** per window is typically the most time-consuming part; using NumPy's `polyfit` or SciPy can help vectorize this.
- Compute the desired features from the output: for classification, one might not use the entire  $F_q(s)$  curves. Instead, features like the Hurst exponent  $H = h(2)$ , the singularity spectrum width  $W = \alpha_{\max} - \alpha_{\min}$ , or specific  $h(q)$  values can be used. In a recent example, fractal dimension coefficients from multiple methods (organized as feature matrices) were used to train a classifier, achieving high accuracy <sup>20</sup> . With MFDFA, one can similarly construct a feature set per audio (e.g. a vector of  $h(q)$  for several  $q$  values).
- **Batch processing considerations:** MFDFA is computationally heavier than simpler FD methods. To streamline, one can predefine the window sizes and use fast array operations. Parallel processing (per audio file) is another way, given each file's analysis is independent. Also, limit the polynomial order  $m$  (DFA order) – usually  $m=1$  or  $2$  is sufficient – to avoid overfitting local

trends and to keep computation reasonable. Monitor the output for numerical stability (very small  $q(s)$  can underflow if using double precision for large  $N$  and high detrending order).

- Ensure reproducibility by fixing analysis parameters (scales,  $q$  values, detrending order) across the dataset. In summary, with careful parameter choices, MFDFA features can be integrated into the front-end of a detection pipeline, complementing other features. The **pipeline vs. end-to-end** design paradigm in deepfake detection suggests using such handcrafted features in a front-end that feeds into a classifier <sup>21</sup>, which in this case could be a machine learning model that takes fractal metrics as input.

## Higuchi's Fractal Dimension Algorithm

Higuchi's method is a popular **time-domain** technique for estimating the fractal dimension of a waveform directly treating it as a geometric curve <sup>22</sup> <sup>23</sup>. It computes the FD by considering the signal at various subsampling intervals, effectively measuring how the length of the waveform curve scales with observation scale. Higuchi's algorithm is known for producing an accurate FD estimate for time series and has been widely applied to biomedical signals (EEG, ECG) and presumably is well-suited for audio as well <sup>24</sup>. It provides a **global fractal dimension** value for a given signal window, though by sliding the window one can obtain a local FD series. Higuchi's FD usually lies between 1 and 2 for waveforms (1 = smooth line, 2 = extremely rough line), and higher values indicate more complexity/roughness in the signal.

**Algorithm (Higuchi's FD):** Let  $x[1], x[2], \dots, x[N]$  be the audio waveform samples (we assume a uniformly sampled signal). Choose an integer maximal scale  $k_{\max}$  (e.g.  $k_{\max}$  might be on the order of 10% of  $N$  or a fixed value like 8, 16, 64 depending on desired granularity). Then perform the following <sup>25</sup> <sup>26</sup>:

- For each scale parameter  $k = 1, 2, \dots, k_{\max}$ :
- Construct  $k$  new time series (subsequences)  $X_m^{(k)}$  defined by:

$$X_m^{(k)} : x[m], x[m+k], x[m+2k], x[m+3k], \dots$$

for starting offset  $m = 1, 2, \dots, k$ . These are basically the original series sampled every  $k$  points (with different starting phases) <sup>25</sup>.

- For each subsequence  $X_m^{(k)}$ , calculate its *length*  $L_m(k)$ . If  $X_m^{(k)}$  has  $N_m$  points, its length can be defined as the sum of absolute differences (with a normalization for the scaling):

$$L_m(k) = \frac{1}{k} \sum_{i=1}^{N_m-1} |x[m+i k] - x[m+(i-1) k]| \times \frac{N-1}{N_m k}.$$

Here the factor  $\frac{N-1}{N_m k}$  is a normalization as given by Higuchi to adjust for the fact that different  $m$  yield different subsequence lengths <sup>27</sup> <sup>28</sup>. Intuitively,  $L_m(k)$  represents the average "curve length" of the waveform when sampled every  $k$  points. (The precise formula may vary slightly in literature; the above is one commonly used form.)

- Compute the average length for scale  $k$  by averaging over all starting positions:

$$L(k) = \frac{1}{k} \sum_{m=1}^k L_m(k),$$

(where the factor  $1/k$  may be part of normalization as well) <sup>26</sup>. Now  $L(k)$  represents the *expected* polyline length of the waveform at resolution  $k$ .

- After computing  $L(k)$  for each  $k = 1, \dots, k_{\max}$ , determine the relationship of  $L(k)$  vs.  $k$ . For a fractal curve, we expect  $L(k)$  to decrease as  $k$  increases (coarser sampling yields a shorter measured curve). In fact, Higuchi showed that  $L(k)$  often follows a power-law:  $L(k) \propto k^{-D}$ , where  $D$  is the fractal dimension <sup>29</sup>. Therefore, we can estimate  $D$  by fitting a line to the log-log plot of  $L(k)$  versus  $k$ . Specifically, perform a linear regression of  $\ln L(k)$  on  $\ln(1/k)$  (or equivalently  $\ln L(k)$  vs  $\ln k$ ), for  $k = 1 \dots k_{\max}$ . The slope of this line is  $-D$  (negative, since  $L(k)$  decreases with  $k$ ), so the *Higuchi fractal dimension* is:

$$D_{\text{Higuchi}} = -\text{slope}\{\ln L(k) \text{ vs } \ln k\}.$$

In practice, one might ignore the smallest scales if the signal has noise, or fit over a certain linear range. The resulting  $D_{\text{Higuchi}}$  will be a value between 1 and 2 for typical audio waveforms (closer to 1 for smoother signals, higher for rough/noisy signals). Higuchi's method is noted for giving accurate FD estimates even on relatively short data, and it was found to be the most *linear and accurate* among time-domain FD estimators in a comparative study <sup>30</sup> <sup>24</sup>.

**Preprocessing:** Higuchi's algorithm works directly on the raw waveform values, and it inherently normalizes by the time step  $k$ . **No explicit preprocessing is strictly required;** even a DC offset is eliminated by the differencing in the length calculation. However, a few considerations can improve results: - It can be useful to **normalize amplitude** ranges across samples (e.g. scale each audio to the same variance or to  $[-1, 1]$ ). While fractal dimension is theoretically invariant to linear amplitude scaling (scaling  $x$  by a constant factors both  $L(k)$  and differences, which cancels out in the slope), extreme differences in amplitude scale might affect numerical stability or interactions with any thresholding. Normalizing ensures one audio's FD isn't biased by having, say, a very low amplitude that's dominated by quantization noise. - **Denoising:** If an audio contains a lot of high-frequency noise, Higuchi's method will measure that as additional roughness (increasing  $L(k)$  at small  $k$  and thus raising the FD). Depending on the application, this might be desirable (noise is part of the signal's complexity) or not (if the noise is an extrinsic artifact like background hum). For deepfake detection, differences in *residual noise* may be a key tell – deepfakes might have either an overly clean background or unnatural artifacts. Thus, one could analyze both the original waveform and perhaps a filtered version. As a best practice, one might apply a mild low-pass filter to focus on the speech band (e.g. 50 Hz – 7 kHz) to remove extreme out-of-band noise that isn't informative. - **Parameter choice:** Decide on  $k_{\max}$ . If the audio segment length is  $N$ ,  $k_{\max}$  can be a fraction of  $N$  (common choices are  $N/10$  or  $N/20$ ) or a fixed number (like 64 or 128) if you want to limit computation. Too large  $k_{\max}$  (close to  $N$ ) means very few points in subsequences, reducing accuracy at those scales. Too small  $k_{\max}$  focuses only on fine scales. For example, if analyzing fractality up to ~50 ms scale on a 16 kHz audio, one might use  $k_{\max} = 800$  (since 800 samples  $\approx$  50 ms). In practice, some experimentation is needed; Esteller *et al.* (2001) used window lengths around 250 points for EEG (1.25 s at 200 Hz) and achieved good results with Higuchi's method <sup>31</sup> <sup>32</sup>.

**Global vs. local:** Higuchi's method yields a **single FD value for the data window** it is applied to. If you feed it a whole utterance, you get that utterance's overall fractal dimension. If instead you slide a window (say 100 ms or 1 s windows) and compute Higuchi FD for each, you obtain a time series of fractal dimension, which can be interpreted as a *local complexity measure*. This local approach has been used in segmentation tasks – e.g. **fractal dimension segmentation** of speech, where FD changes can indicate transitions between speech sounds of different texture <sup>33</sup> <sup>34</sup>. For deepfake detection, one could either use a **global FD** per utterance (to see if an entire fake utterance tends to have a different complexity than a real one) or use **local FD features** (e.g. statistics of the FD over frames, or even feeding the sequence of FD values into an LSTM or transformer classifier). If using local values, it's important to choose a window length that is long enough to get a stable FD estimate but short enough to capture time-variations; a few hundred samples (tens of milliseconds) might be too short for reliable

FD, whereas ~0.5–1 second windows might smooth over microvariation. A compromise could be ~100 ms windows with overlap – this can capture phoneme-level changes in speech while still giving Higuchi’s algorithm enough points to work with.

Notably, in comparisons, **Higuchi’s FD** was found to be more accurate and less sensitive to noise and window length than Katz’s or Petrosian’s in reproducing theoretical fractal dimensions <sup>30</sup> <sup>24</sup> . Thus, Higuchi’s is often a preferred method for precise FD estimation.

**Relevance to deepfake detection:** Fractal dimension can serve as a discriminative feature reflecting the **complexity of the waveform**. A genuine speech signal might have a different fractal complexity than a synthesized one. For instance, if a generative model produces speech with overly smooth transitions (lack of jitter, perfectly interpolated frames), the waveform could appear smoother (lower FD ~ closer to 1). Conversely, if a deepfake introduces odd high-frequency artifacts or a choppy voice due to model imperfections, it might raise the FD. Higuchi’s method, because it directly measures curve length at multiple scales, will **quantify these differences**. It is sensitive to subtle changes in waveform shape: e.g., **glottal pulses** in natural speech create a certain roughness at the pitch period scale, whereas a vocoder-synthesized voice might smooth or quantize those pulses differently. Higuchi’s FD could capture such differences in the 5–15 ms range (for typical pitch periods). Additionally, **residual breath noise or micro-tremors** in real speech contribute to complexity – deepfake systems might not accurately reproduce those, leading to complexity gaps.

In EEG research, although Higuchi gave the most accurate FD values, interestingly **Katz’s FD** was reported to be more *consistent in discriminating states* despite being less linear <sup>35</sup> <sup>36</sup> . By analogy, it might be that while Higuchi is mathematically precise, another method could yield a larger separation between classes. Nonetheless, Higuchi FD is a strong baseline feature. It can be used alongside others to build a rich feature set. For example, one could compute Higuchi FD, Katz FD, etc., and use all of them as input features. In fact, recent audio classification work combined multiple FD measures: one study extracted *four different fractal dimensions* (likely including Higuchi’s) and organized them into a feature matrix to feed a neural network, achieving high accuracy in a classification task <sup>37</sup> .

**Implementation in a pipeline:** Higuchi’s method is straightforward to implement in Python: - **Sampling rate:** Ensure a consistent sampling rate, as fractal dimension should be computed on comparable time-scales. If using time in samples (with sample index as the time unit), it’s fine as long as all audio have same rate; otherwise, if audio rates differ, consider converting to a standard rate (e.g. 16 kHz) before FD computation. - **Vectorization:** The algorithm involves nested loops (over  $k$  and  $m$  ). However, one can vectorize parts of it. For example, using NumPy to compute differences for each  $k$  can speed it up, or restrict to a reasonable  $k_{\max}$  . There are also existing implementations (like `Higuchi_FD` in some libraries or online code) that can be utilized. The `AntroPy` library (by Vallat) provides a `higuchi_fd` function for one-dimensional data, which can be leveraged for batch processing. - **Batch processing:** If computing for many files, you can loop through files, and inside each, loop through  $k$  . The complexity is  $O(N \times k_{\max})$  per signal. For a typical utterance of a few seconds (say  $N=50,000$  samples) and  $k_{\max}=100$ , this is on the order of 5 million operations – easily done in under a second in Python with optimizations. In batch, this is quite tractable. If doing sliding-window FD, the cost multiplies by the number of windows; in that case, using a *rolling computation* (reusing computations between overlapping windows) could help, but often one might just recompute since the window length isn’t huge. - **Output:** Higuchi FD yields a single floating-point number per segment. In a pipeline, these values could be used directly as features. For example, you could have one feature “Higuchi\_FD” per audio. Alternatively, you might use statistics of local FD (mean, variance across frames) as features. - **Best practices:** Ensure the data segment you analyze has enough samples (a very short clip may give unreliable FD). Also, avoid extremely large  $k_{\max}$  relative to segment

length – Higuchi’s method works best when  $L(k)$  can be averaged over many segments (i.e.  $N$  should be much larger than  $k_{\max}$ ). If your audio files are long, you might break them into several chunks and compute FD for each chunk to see if variability exists. - **Combining with other features:** FD features like Higuchi’s can complement spectral features (MFCCs, spectrogram-based) by providing a direct measure of waveform complexity. They are also complementary to voice-quality features like jitter/shimmer (which are conceptually related to fractal roughness). In a classifier, using Higuchi’s FD alongside MFCC-based embeddings could improve discrimination between real and fake. Notably, fractal features can be used as input to classical machine learning or even to deep models (as in the example of forming a matrix of FD coefficients for a CNN/transformer <sup>37</sup>).

## Katz’s Fractal Dimension Algorithm

Katz’s method is an older but intuitive approach that computes fractal dimension from the *length* of a waveform and its geometric “span.” It treats the time series as a polyline in the 2D plane (time on one axis and amplitude on the other) and compares the actual length of that curve to the distance between its endpoints <sup>38</sup> <sup>39</sup>. The idea is that a more convoluted (highly wiggly) curve will be much longer than the straight-line distance between start and end, leading to a higher fractal dimension. Katz’s FD is simple and fast to compute, yielding a single global value per signal. It does not involve multiscale analysis like Higuchi’s; rather, it gives a one-shot estimate of dimension based on an “effective length versus diameter” ratio.

**Algorithm (Katz’s FD):** For a given waveform  $x[1 \dots N]$  sampled at a constant rate: 1. Compute the total *curve length*  $L$  of the signal path. Katz’s original definition uses the sum of Euclidean distances between successive points. If the time increment between samples is  $\Delta t$  (e.g.  $\Delta t = 1$  if we measure time in sample indices), then for each adjacent pair of points  $(i, x[i])$  and  $(i + 1, x[i + 1])$ , the Euclidean distance is:

$$d_i = \sqrt{(\Delta t)^2 + (x[i + 1] - x[i])^2}.$$

If using sample index as the time unit,  $\Delta t = 1$ . For small differences, one could approximate  $d_i \approx |x[i + 1] - x[i]|$  if  $\Delta t$  is unity, but the Euclidean formula is more general (and ensures consistency if one changes sampling rate units). Now sum these distances:

$$L = \sum_{i=1}^{N-1} d_i.$$

This  $L$  is the total length of the waveform curve from start to finish <sup>40</sup> <sup>41</sup>.

1. Compute the *diameter* of the curve, defined by Katz as the greatest distance between the first point and any other point on the curve <sup>42</sup>. In practice, this is often taken as the straight-line distance from the first sample to the farthest sample:

$$d = \max_{2 \leq j \leq N} \sqrt{((j - 1)\Delta t)^2 + (x[j] - x[1])^2}.$$

Here  $(j - 1)\Delta t$  is the time difference from the first point to point  $j$ , and  $(x[j] - x[1])$  is the amplitude difference. Essentially, you find the point in the time series that is *farthest* in Euclidean distance from the start. Often, this will be either the last point or a point with an extreme amplitude. (If the signal oscillates, the farthest might be an interior point with a large amplitude swing.) Katz’s method uses this  $d$  as a measure of the “overall size” of the curve <sup>43</sup>.

2. Compute the average step size  $a$  as the total length  $L$  divided by the number of steps (points) in the curve. If there are  $N$  points, there are  $N-1$  intervals, so:

$$a = \frac{L}{N - 1}.$$

This represents the *average distance between successive points* (the “yardstick” length) <sup>44</sup>. Katz introduced this normalization to make the FD dimensionless and unit-independent <sup>39</sup>. Without normalization, FD estimates could change if one measured time in seconds vs. milliseconds; using  $a$  mitigates that issue by expressing lengths relative to an intrinsic scale of the data.

3. Finally, **Katz’s fractal dimension** is defined as:

$$D_{\text{Katz}} = \frac{\log_{10}(L/a)}{\log_{10}(d/a)},$$

where  $L/a$  can be interpreted as the number of steps of length  $a$  that make up the curve, and  $d/a$  as the number of such steps in the “diameter” <sup>45</sup>. This formula can be simplified: note  $L/a = (L/(N-1))^{N-1} L = N-1$  (since  $a = L/(N-1)$  by definition), so  $\log_{10}(L/a) = \log_{10}(N-1)$ . Thus one can rewrite it as

$$D_{\text{Katz}} = \frac{\log_{10}(N - 1)}{\log_{10}(d/a)},$$

which is sometimes seen in literature <sup>46</sup>. Another rearrangement yields  $D = \frac{\log_{10}(L/a)}{\log_{10}(d/a)} = \frac{\log_{10}L - \log_{10}a}{\log_{10}d - \log_{10}a}$ . Katz originally described it intuitively as comparing “the actual number of units composing the curve” to “the minimum number of units (of length  $d$ ) required to reproduce a pattern of the same extent” <sup>38</sup> <sup>39</sup>. The use of  $a$  makes the measure independent of the specific units of measurement <sup>47</sup>.

In summary, Katz’s FD essentially measures how efficiently the waveform extends through space: if the waveform is a straight line, then  $L \approx d$  and the FD will be close to 1. If the waveform is very convoluted (long length  $L$  relative to overall span  $d$ ), then  $D$  increases. Katz’s formula will always yield a value  $\geq 1.0$ . Typical values for signals range maybe up to ~1.5 or 1.6 in practice (Katz reported random-walk ECG-like waveforms around 1.15, etc., since extremely jagged signals might increase it a bit but it tends to saturate) <sup>48</sup>.

**Preprocessing:** Katz’s FD is very easy to compute and does not explicitly require preprocessing of the signal. However, a few notes: - **Uniform sampling:** Ensure the time difference  $\Delta t$  is consistent. If comparing signals of different sampling rates, it’s crucial to normalize time units. The formula as given implicitly assumes equal time steps. If all audio are at the same sampling rate (which they should be in a controlled pipeline), you can set  $\Delta t = 1$  (one index step) and the units cancel out in the ratio. If audio lengths differ significantly,  $N$  differs but that’s fine –  $D$  should be comparable as it’s unitless. - **DC offset:** A constant offset in  $x$  does not affect  $L$  (differences cancel) or  $d$  (since only differences matter), so it doesn’t matter. No need to remove mean specifically for Katz (unlike in MFDFA where integration would emphasize it). - **Amplitude normalization:** Because  $D$  uses a ratio, scaling the entire amplitude by a constant  $c$  will *not* change  $D$ . To see this: if  $x$  is scaled by  $c$ , then  $L$  scales roughly by  $c$  (each  $d_i$  scales by  $c$  in the vertical part), and  $d$  scales by  $c$  as well (as  $x[j]-x[1]$  gets  $c$ ). Then  $d/a$  remains same because both  $d$  and  $L$  (hence  $a$ ) scale by  $c$ . Thus, Katz’s FD is invariant to linear amplitude scaling. This is convenient – it means you don’t necessarily have to volume-normalize the files; a quiet and loud utterance (if shape is same) would yield



the same FD. However, if signals have different noise floors or sample quantization, very small amplitude signals could be influenced by relative measurement error. It's still wise to bring signals to a standard amplitude range (say peak normalize or standardize variance) for numerical stability. - **Noise/thresholding:** Katz's method will count every small zigzag. If the audio has high-frequency noise,  $SL$  will increase (lots of small up-down changes) while  $d$  likely remains about the same, thus raising  $FD$ . In some cases, this could exaggerate differences due to noise. If we want  $FD$  to reflect more the *macro-structure* of the waveform, one could pre-smooth the signal slightly (e.g. a low-pass filter or moving average) to remove tiny fluctuations. On the other hand, those tiny fluctuations might be exactly what differentiates a fake from a real. For instance, *background hiss or quantization noise* might be present in a deepfake but not in a clean studio-recorded real speech, or vice versa real speech might have more physiological micro-perturbations. So the decision to filter noise should align with what aspect you think is indicative of spoofing. Katz's FD in its raw form will include all fluctuations. A compromise approach could be to compute it on a **downsampled version** of the waveform (say, downsample by a factor of 2 or 4) to inherently ignore changes above a certain frequency. This effectively sets a threshold on the smallest scale considered. Katz's formula doesn't have an explicit "scale" parameter, but downsampling the data reduces the total number of points and smooths the curve. - **Segmentation:** If an audio has long silences or very low-energy parts, Katz's FD might be dominated by the active portion but still count the whole duration in  $d$ . For example, if the first part of the file is silence and later part is speech, the farthest point might be at the end (in time), and the amplitude difference from start might be small (start is silence level, far end might be back to silence after speech). Then  $d$  could be mostly horizontal distance, and  $SL$  includes the speech fluctuations. This might yield a certain  $FD$ . If you remove the silence and just analyze the speech segment,  $N$  and  $d$  become smaller (the utterance span shrinks), which could change  $FD$ . To maintain consistency, it's often better to analyze only the voiced portion for Katz's FD in speech. This means applying a simple VAD or threshold to cut off leading/trailing silences. This way,  $d$  truly reflects the span of the speech activity rather than the recording length.

**Global vs. local:** Katz's method is inherently global for the segment of data you feed into it. It yields one number for that segment. If you want a local measure, you can apply Katz's FD to short overlapping frames. However, one must be careful – Katz's definition uses the **first point as a reference** for  $d$  (distance from first point). In a sliding window context, you would compute  $d$  within each window relative to that window's first sample. This means Katz's FD on a window depends on the window's baseline. For a highly oscillatory waveform, if the window happens to start at a peak or trough, the  $d$  might be smaller or larger. Typically, if using short windows, one might modify Katz's method slightly: e.g., define  $d$  as the maximum distance between *any two points* in the window (the "span" of that segment) rather than necessarily first point to farthest. However, the original method specifically used the first point to avoid ambiguity and ensure a unique measure <sup>42</sup>. In practice, for local analysis, this nuance might introduce variability. Nonetheless, using Katz's FD on, say, each 1-second chunk of speech can yield a time series showing how complexity changes. Researchers have found Katz's FD to vary with physiological state in EEG over time <sup>36</sup>, so by analogy it could vary across phonetic events or speech sections. But the dynamic range of Katz's FD is not large (often ~1 to 1.3 as noted in experiments <sup>49</sup>), so changes are subtle.

Often, Katz's FD is used as a **single feature** describing the whole signal. For deepfake detection, one could compute Katz's FD for the entire utterance. If the dataset has many utterances, you could observe if fakes cluster at a different FD value than reals. One advantage is that Katz's FD is extremely fast to compute, so it's feasible to use it on the fly even in real-time scenarios.

**Relevance to deepfake vs. genuine speech:** Katz's fractal dimension captures the ratio of total "travelled" waveform length to the overall size. Intuitively, **human speech** tends to have a lot of fine detail (rapid oscillations due to pitch, formant vibrations, bursts in fricatives, etc.), which increases  $SL$ ,

whereas the overall amplitude range and duration ( $\Delta d$  in combination with time) might not increase proportionally. A well-trained **deepfake** might actually produce speech that is *too perfect* or smooth in certain areas – for example, less microvariation in pitch or less jitter noise – potentially leading to a slightly shorter total length  $L$  for the same overall  $\Delta d$ . This would yield a **lower Katz FD**. Conversely, some deepfakes might introduce high-frequency artifacts (like subtle digital noise or robotic buzzing) that increase  $L$  disproportionately, yielding a **higher Katz FD** than natural. The actual direction of difference can depend on the generation method. The key is that **Katz FD provides a single metric of signal complexity** that could differentiate the two. In scenarios outside deepfakes, Katz's FD has been used to distinguish physiological states (e.g. Katz FD separated pre-seizure vs. seizure EEG segments in a consistent way) <sup>36</sup>. In that study, although Higuchi's method was more accurate in absolute terms, Katz's FD showed the most consistent discrimination between brain states <sup>50</sup>. This suggests Katz's formula might amplify certain pattern differences making classes more separable, even if it's less "linear." Applying this insight, one could expect Katz's FD to be a valuable **feature for classification**: e.g., real speech might consistently show Katz FD around (hypothetically)  $1.10 \pm 0.02$ , whereas certain deepfake voices might show  $1.05 \pm 0.02$ , forming two distributions that a classifier can learn to tell apart. Even if the means differ by only 0.05, with enough data that can be significant (especially combined with other features).

In practice, Katz's FD **alone** might not perfectly discriminate all fakes, but as part of a feature ensemble it adds a useful dimension (literally) of information. It's computationally cheap, so there is little downside to including it. Moreover, it has an intuitive physical meaning, which can be useful for explainability – e.g., "the fake speech had a fractal dimension closer to 1, indicating a smoother waveform, whereas genuine speech had a rougher profile (higher dimension)."

**Implementation in pipeline:** Katz's algorithm is extremely fast ( $O(N)$  for computing length plus a pass to find farthest point). For batch processing: - Simply iterate through each audio, compute  $L$  and  $\Delta d$  and then  $\Delta d$ . Using NumPy, you can vectorize the distance sum: e.g. `diffs = np.diff(x)` to get successive differences, then `L = np.sum(np.sqrt((diffs_time)^2 + (diffs_amp)^2))`. Here `diffs_time` could be 1 for all steps if time step = 1. Actually, one can simplify: if  $\Delta t = 1$ ,  $\Delta d_i = \sqrt{1 + (\Delta x_i)^2}$ . Summing many square roots is not easily vectorized without computing each element, but using numpy it's still in C loop, so fine. - To get  $\Delta d$ , you could compute distances of all points from the first point. But a quicker approach: *assuming the main contribution to distance is horizontal (time axis) when  $j$  is far from 1*, the farthest point often ends up being the *last* point (distance  $\sqrt{(N-1)^2 + (x[N]-x[1])^2}$ ) or a point with extreme amplitude. To be safe, you can compute `dist = (np.arange(N)-0)**2 + (x - x[0])**2` and take sqrt of max of that. That's vectorized (squaring arrays) and efficient. Or simply loop, since  $N$  isn't huge typically. - Put the result into a feature array. If combining features, you may create a feature vector like `[Higuchi_FD, Katz_FD, Petrosian_FD, ...]` for each sample. In one study, an image was formed out of multiple FD measures for input to a CNN <sup>37</sup> – they effectively treated different FD algorithms as different "channels" of information. You could do similarly: e.g., compute Katz FD on various sub-bands of the audio or on frames and arrange that into a feature matrix. - **Parameter notes:** Katz's FD has essentially no tunable parameter except how you define  $\Delta t$ . If all signals same sampling rate, just keep  $\Delta t = 1$  (one sample). If using actual time units (seconds), one could set  $\Delta t = 1/\text{Fs}$  in seconds, but then  $N$  is large and  $\Delta d$  gets measured in seconds. The ratio will cancel the unit, so it doesn't matter – just ensure consistency. It's simpler to use sample index units. - Because Katz's FD values may occupy a narrow range (say 1 to ~1.3 for most speech signals), when feeding into some machine learning models, one might do feature scaling (e.g. standardize to zero mean, unit variance across the training set). This will emphasize small differences. - For **real-time use**, Katz's method is well-suited: you could stream audio and update the FD on the fly in a sliding manner if needed, due to its low complexity. For offline batch (e.g. ASVspoof challenge evaluation), it's trivial to compute on thousands of utterances quickly. - **Combining with MFCCs or others:** If you already have MFCC or

spectrogram-based features, adding fractal features like Katz's is easy – just concatenate to the feature vector. They capture complementary info (MFCCs reflect spectral envelope shapes, fractal FDs reflect waveform geometry complexity).

In summary, Katz's fractal dimension is a quick way to get a single-value descriptor of waveform complexity. It eliminates units and yields a dimensionless number characterizing the waveform's shape. This can be a useful discriminator between real and fake speech, especially if fake speech is either *too simple* (yielding lower FD) or *has irregular artifacts* (possibly yielding a distinct FD signature) compared to real speech.

## Petrosian's Fractal Dimension Algorithm

Petrosian's method offers a **fast, simplified estimate of fractal dimension** by counting the number of sign changes in a signal <sup>51</sup>. Instead of directly measuring lengths or multi-scale lengths, Petrosian FD focuses on how often a time series switches "direction," which is a proxy for complexity. It was originally introduced for quick analysis of EEG signals by first converting them to a binary sequence <sup>52</sup> <sup>53</sup>. In essence, Petrosian's FD is related to the signal's frequency of oscillation: more oscillatory (more zero-crossings or direction changes) implies a higher fractal dimension, up to a point. The advantage of Petrosian's approach is its computational simplicity – it reduces fractal dimension calculation to a simple count operation. However, it yields a **very limited dynamic range** of values (often very close to 1 for many signals) <sup>54</sup>, making it a coarse measure. It's best thought of as a quick indicator of signal roughness.

**Algorithm (Petrosian's FD):** There are a few variations in how the binary sequence is obtained (Petrosian's 1995 paper presented multiple methods labeled A, B, C, D) <sup>55</sup> <sup>56</sup>. A common approach, which we will use here, is: 1. **Derive a binary sequence from the signal.** One simple way (akin to method C in Petrosian's paper <sup>53</sup>) is: - Compute the first difference of the signal:  $\Delta x[i] = x[i+1] - x[i]$ . - Convert this to a sign sequence  $b[i]$  where  $b[i] = 1$  if  $\Delta x[i] > 0$  (the signal is rising) and  $b[i] = 0$  if  $\Delta x[i] \leq 0$  (the signal is falling or steady). - This yields a binary string of length  $N - 1$  indicating up/down moves. - Alternatively, one can directly look at the original signal: e.g. set  $b[i] = 1$  if  $x[i] > x[i-1]$ , 0 otherwise (which is effectively the same). Some methods use thresholding around the mean or mean  $\pm$  std to create the binary series <sup>57</sup>, but using sign of derivative is common and captures zero-crossings effectively <sup>58</sup>.

1. **Count the number of sign changes** (also called *turning points* or dissimilar pairs) in the binary sequence. Essentially, this is counting how many times the binary sequence flips from 0 to 1 or 1 to 0. If we denote  $N_\delta$  as the number of sign changes, it can be calculated by:

$$N_\delta = \sum_{i=1}^{N-2} \mathbf{1}_{\{b[i] \neq b[i+1]\}}$$

where the indicator is 1 when a change occurs and 0 otherwise. This is effectively the number of **zero-crossings** in the *derivative* of the signal (i.e., how often the signal's slope changes sign) <sup>58</sup>.

Note: if the method uses the signal itself (like method A which thresholds around the mean), then  $N_\delta$  would correspond to zero-crossings of the signal around that threshold. But using the sign of differences,  $N_\delta$  counts oscillations regardless of baseline shifts.

2. **Compute Petrosian's fractal dimension:** Petrosian proposed an empirical formula to relate  $N_\delta$  to fractal dimension. One widely used formula is <sup>59</sup>:

$$D_{\text{Petrosian}} = \frac{\log_{10}(N)}{\log_{10}(N) + \log_{10}\left(\frac{N}{N + 0.4N_{\delta}}\right)},$$

where  $N$  is the length of the original time series (number of samples) and  $N_{\delta}$  is the number of sign changes in the chosen binary representation. This formula is a slight refinement – the 0.4 factor is an empirical constant Petrosian included to adjust the FD estimate <sup>60</sup> <sup>58</sup>. Essentially, when  $N_{\delta}$  is large (many sign changes), the term  $N/(N + 0.4N_{\delta})$  is significantly less than 1, increasing the denominator and keeping  $D$  closer to 1. When  $N_{\delta}$  is small (signal hardly changes direction), the fraction  $N/(N + 0.4N_{\delta})$  is near 1, so the denominator  $\log_{10}(N) + \log_{10}(\text{something} \approx 1)$  is  $\approx \log_{10}(N)$ , making  $D \approx \frac{\log_{10}(N)}{\log_{10}(N)} = 1$ . In the extreme, a perfectly monotonic signal (no sign changes) gives  $N_{\delta}=0$ , and the formula yields  $D=1$  (as expected, a straight line is dimension 1). For a very erratic signal (lots of sign flips),  $D$  will increase slightly above 1. In practice, Petrosian FD values for most natural signals tend to be in a narrow range (e.g. 1.0 to 1.2) <sup>61</sup> <sup>62</sup>. It basically measures “oscillatory richness.”

*Note:* In some sources, a simpler formula without the 0.4 is given:  $D = \frac{\log_{10}(N)}{\log_{10}(N/N_{\delta})}$ . However, the version with 0.4 is widely cited (likely originating from Petrosian’s work or subsequent calibrations) and implemented in libraries (e.g. `entropy.petrosian_fd` uses the 0.4 factor <sup>59</sup>).

Thus, Petrosian’s algorithm reduces the fractal dimension problem to computing  $N_{\delta}$  efficiently (which is  $O(N)$ ) and plugging into the formula. The result is dimensionless. If, for example, the signal is a pure sine wave over many cycles, each half-cycle causes a sign change in the derivative (peaks to troughs), so  $N_{\delta}$  roughly twice the number of oscillations. If the signal is random white noise,  $N_{\delta}$  will be very high (almost every sample flips sign relative to previous), but notice  $D$  saturates because as  $N_{\delta}$  approaches  $N$ , the formula asymptotically gives a certain upper limit (a very large  $N_{\delta}$  yields  $\log_{10}(N/(N+0.4N_{\delta})) \approx \log_{10}(1/(0.4N_{\delta}/N + 1)) \approx \log_{10}(1/(\text{something} \approx \text{large}))$  which limits the denominator). Indeed, Petrosian FD tends to saturate around  $\sim 1.4$  for extremely noisy signals and doesn’t go to 2 <sup>61</sup>.

**Preprocessing:** The main preprocessing step is **binarization** of the signal. Depending on the method: - If using the **sign of differences** (recommended for audio), you effectively detrend the signal by looking at changes, so DC offset is irrelevant. No further preprocessing needed other than perhaps smoothing as discussed next. - If one used a threshold method (like “Method A” from Petrosian: assign 1 if above mean, 0 if below mean), then a preliminary step is computing the mean or threshold. But that method might be less sensitive to brief fluctuations and more to overall level crossing – which might not capture fine detail as well. - **Noise and threshold:** Petrosian’s method in raw form will count even tiny sign changes due to noise. If your audio has high-frequency noise,  $N_{\delta}$  will be large. If that noise is present in both real and fake data similarly, it just raises all FD values by similar amount (not an issue for classification except reducing dynamic range further). If one wants to focus on meaningful fluctuations, one can introduce a **threshold** for what constitutes a change. For example, only count a sign change if the amplitude change exceeds some epsilon. Petrosian did propose a variant (Method D) where differences smaller than a certain threshold (like a fraction of standard deviation) are treated as no change <sup>63</sup> <sup>64</sup>. In audio, one might set a small threshold to ignore sign flips caused by very low-level noise. This can stabilize  $N_{\delta}$  against background hiss. Another approach is to low-pass filter or downsample the signal slightly (removing very high-frequency components) before computing Petrosian FD. - **Segmentation (silence handling):** If the audio has long flat sections (silence), these contribute  $N_{\delta} = 0$  in those intervals, but the formula’s use of full  $N$  still counts them. In a long silence, there are no sign changes, so  $D$  will be  $\sim 1$ . If an utterance has speech then silence, the overall

$\delta$  might be diluted by the silence. If focusing on voice part, it's advisable to remove leading/trailing silences or treat them separately. Perhaps compute FD on voiced segments only. Alternatively, you could calculate FD over the entire signal including silence – if deepfakes often have different silence (like perfectly zero vs. real recordings having slight noise), that difference could also reflect in  $\delta$ . But generally silence will lower FD (since no changes), and many deepfakes might insert absolute silence which would yield exactly 1.0 FD for that portion. A strategy: calculate Petrosian FD on just the active speech portion of the file for a fair comparison.

**Global vs. local:** Like Katz's, Petrosian's method yields a single number per analyzed segment. You can apply it to an entire utterance or to short windows. Because Petrosian FD reacts mainly to frequency of sign flips, a short window covering at least a couple of oscillations is needed for a meaningful value. For example, a 50 ms window of voiced speech (pitch period ~10 ms) might have a few sign changes in the derivative, giving some FD value slightly >1. A 1 second window might have many, but Petrosian's formula normalizes by  $\log_{10}(N)$  so there is some scaling accounted for. One interesting aspect: **Petrosian FD doesn't change with signal length if the rate of zero-crossings remains proportionate.** For a stationary oscillatory signal, doubling  $N$  and proportionally doubling  $\delta$  yields nearly the same FD (the formula would give identical result if  $\delta/N$  stays constant). So Petrosian's FD is somewhat *stationarity-friendly* – you can compute it on chunks and get similar values, as long as the nature of the oscillations in that chunk is similar. Therefore, using a long global window vs. an average of local windows might not differ much, except for capturing nonstationary changes. If one expects that *during certain phonemes* a fake might behave differently (say, in unvoiced fricatives maybe a deepfake has a more periodic artifact or something), one could look at Petrosian FD per phoneme or per short frame. But because the range is tiny (say one frame yields 1.03 vs another 1.05), one might instead use the **global Petrosian FD** as a single feature. Perhaps more useful: Petrosian FD on different **frequency subbands**. In speech, high-frequency components (e.g. in the 4–8 kHz band) could have different roughness than low-frequency. A variant approach could be to filter the audio into bands and compute Petrosian FD for each – essentially measuring the fractal dimension of the waveform's envelope in each band. This can reveal if a deepfake fails to produce correct high-frequency “noisiness,” etc. However, that's an advanced usage beyond the basic method.

**Relevance to deepfake detection:** Petrosian's FD is basically a measure of **signal oscillation complexity** in a yes/no sense. For real vs. fake speech: - Real speech typically has a certain natural **zero-crossing rate** pattern. For example, voiced speech has a quasi-periodic structure (zero-crossings influenced by pitch and formant shapes), and unvoiced speech (fricatives) has a higher zero-crossing rate (being noise-like). A deepfake might inadvertently alter these patterns (some TTS systems historically had difficulty generating truly random-like high-frequency noise, leading to more voiced-like structure even in fricatives). If a deepfake's fricatives are too “clean” or tonal, the zero-crossing count might be lower than a real fricative which is noisy. Conversely, if a deepfake injects odd rapid fluctuations, it could raise zero-crossings. - Because Petrosian FD is so closely linked to zero-crossing counts, it might correlate with classic features like **ZCR (zero-crossing rate)**. In fact, ignoring the log formula,  $\delta/N$  is essentially the fraction of sign changes per sample, i.e. half the zero-crossing rate (since each zero-crossing yields two sign changes in derivative). Thus Petrosian FD is a monotonic function of ZCR. We know ZCR has been used in audio analysis (e.g. voiced vs unvoiced classification). For deepfake detection, differences in ZCR distribution between real and fake have been reported in some cases. Petrosian FD packages that into a single metric with some normalization. - Petrosian FD tends to **saturate**, meaning it won't drastically differentiate very rough signals – multiple signals with lots of fluctuations will all yield FD near ~1.1. In Esteller's comparison, all variations of Petrosian's method yielded FD between ~1.01 and 1.06 across a wide range of synthetic fractal signals <sup>49</sup> <sup>65</sup>. This limited dynamic range could make it less sensitive for subtle differences. However, it also means it's **robust to noise** in a way – moderate noise doesn't blow the metric out; it stays around ~1.x. Indeed, the study found Petrosian's method had almost no dependence on window length or noise in that small

range <sup>65</sup> <sup>36</sup>. This stability could be beneficial: if real and fake differ consistently by, say, 0.02 in Petrosian FD on average, that difference might hold across conditions. - Specifically, if deepfake speech lacks the fine chaotic elements of genuine speech, we might find: **real speech Petrosian FD > fake speech Petrosian FD** (because real has more frequent small fluctuations, perhaps due to natural noise in the voice or recording). If the deepfake introduces odd high-frequency artifacts, it might swing the other way, but many current TTS/VC systems aim for smoothness, which likely reduces the number of rapid sign changes (besides those necessary for the speech content). - Petrosian FD could also capture differences in *voicing behavior*: For example, the transitions between voiced and unvoiced sounds in natural speech might produce certain patterns of sign changes, whereas a deepfake might handle these transitions differently (some early deepfakes had unnatural prosody or too-abrupt transitions). - Because this measure is so simple, it may not be a sole strong discriminator, but it's a good **complementary feature**. It provides a quick snapshot of "how oscillatory" the waveform is. For instance, if one finds that an impersonator (fake) tends to produce smoother waveforms, all Petrosian FD values for fakes might cluster a bit lower than reals. It's not guaranteed for all cases, but it's easy to check empirically on a development set.

**Implementation in pipeline:** Petrosian's FD is trivial to implement: - Compute sign changes: e.g. `diff = np.diff(x)` then `signs = diff > 0` (boolean array), then `N_delta = np.sum(signs[:-1] != signs[1:])`. This gives the count of changes from sample to sample. This is vectorized and extremely fast (O(N)). - Plug into formula. The formula involves logarithms; ensure to use float for N to avoid integer division. Also, handle  $N_{\Delta}=0$  (if absolutely no changes, the formula would give 1 as limit – you can hardcode that output to 1 if  $N_{\Delta}=0$  to avoid log10 of 1/1). - One can also use existing functions: e.g. Raphael Vallat's `entropy.petrosian_fd` (from the Antropy library) implements this and can handle ND arrays in one go <sup>66</sup>. - In batch, just loop over files or use a comprehension to get all values. It's negligible in computation time compared to reading the file from disk – millions of samples per second can be processed for sign changes easily. - **Parameterization:** The only parameter is whether to threshold differences. If you suspect the need, you might set an epsilon relative to the signal's standard deviation. For example, ignore changes smaller than 5% of std. However, choosing this threshold objectively is tricky and could remove meaningful details (especially since we want to detect differences). Many implementations skip thresholding (the 0.4 factor in formula is effectively a generic compensator). - If focusing only on active speech, you might do: Petrosian FD on voiced frames and on unvoiced frames separately to see differences. Or more simply, exclude long silences as mentioned. - **Integration with other features:** Because Petrosian FD is correlated with ZCR and frequency content, it might partly overlap with features that a spectrogram-based classifier would pick up. However, including it explicitly can still help a classical ML model or can guide attention in a hybrid system. It's such a low-cost feature that it's worth extracting. - **Multiple segment combination:** If you have local Petrosian FD values (say one per short frame), you could take stats (mean, variance) or percentiles. But given the narrow range, often the mean or median Petrosian FD of an utterance is nearly the same as the single FD computed on the whole utterance. Therefore, computing one per utterance is usually enough.

In summary, Petrosian's method provides a **fast complexity estimate** based on sign-change frequency. Its value in an anti-spoofing pipeline would be as a lightweight indicator of waveform "roughness." Combined with heavier features, it can contribute to a richer feature set. For example, a survey of detection methods might incorporate fractal features like Petrosian FD and report improved robustness <sup>37</sup>. Given its ease of calculation, it's definitely a candidate for inclusion in a batch feature-extraction stage. One can compute Petrosian FD for each audio in the dataset and analyze if fakes and reals show a statistical difference. If they do, even a simple threshold could serve as a quick filter or an input to a larger classifier.

## Box-Counting Fractal Dimension (Minkowski–Bouligand Dimension)

The **box-counting** method is a general technique to estimate the fractal (capacity) dimension of a geometric set by counting how many cells of a given size are needed to cover the set <sup>67</sup> <sup>68</sup>. When applied to a time-series (audio waveform), we interpret the waveform as a **2D curve** in the plane (time vs. amplitude) and attempt to measure its fractal dimension by covering it with grids of various resolutions. Box-counting FD (also known as Minkowski dimension) is conceptually close to the mathematical definition of fractal dimension and is often used for fractals in images. For 1D signals, it can be used but tends to be sensitive to how the grid is scaled on time vs. amplitude axes. Nonetheless, it is an important method to mention, as it directly approximates the Hausdorff dimension by a simple counting procedure <sup>67</sup> <sup>69</sup>.

**Algorithm (Box-Counting FD) for a time series:** 1. **Normalize or set up coordinate system:** Let the time axis span the duration of the audio and the amplitude axis span the range of signal values. For box-counting, it can help to **normalize** both axes to a unit interval or comparable scale. One common approach is to embed the time series in a 2D grid of size  $N \times N$  (with  $N$  a power of 2 for convenience) <sup>70</sup>. For example, suppose we have  $N$  samples; we can consider a square grid of  $N \times N$  where the x-axis index  $i$  (time) runs 1 to  $N$  and y-axis covers amplitude (after mapping amplitudes into  $[1, N]$  through linear scaling). This ensures both dimensions are treated equally in terms of box size. Another approach is to use actual units: e.g. time in seconds and amplitude in, say, normalized units, but then one must decide what constitutes a “square” box (likely one needs to choose an aspect ratio so that a box of side  $\epsilon$  means  $\epsilon$  in time and  $\epsilon$  in amplitude units).

1. **Choose a set of box sizes (scale  $\epsilon$ ):** Typically, one uses a dyadic sequence of box sizes (like dividing the domain by 2, 4, 8, ...). For instance, if working in the normalized  $N \times N$  grid, you might use box side lengths  $\epsilon = N, N/2, N/4, \dots, 1$  (in grid units) <sup>71</sup> <sup>72</sup>. Each box size  $\epsilon$  corresponds to partitioning the domain into  $(N/\epsilon) \times (N/\epsilon)$  grid cells.
2. **Count the number of boxes that contain part of the curve:** For each box size  $\epsilon$ , cover the entire space with a grid of that resolution and count  $N(\epsilon)$  = the number of boxes that intersect the waveform. A box “intersects” the waveform if any segment of the waveform (line between consecutive sample points) passes through that box. Equivalently, you can mark all grid cells that the curve goes through. In practice:
3. For a given  $\epsilon$ , the time axis can be divided into segments of length  $\epsilon$  and amplitude axis into bins of height  $\epsilon$ . Then iterate over each time segment and find the range of the curve within that segment.
4. A simpler approach in 1D: The **yardstick algorithm** (a variant) – measure the length of the curve with steps of size  $\epsilon$  (like the divider method) <sup>73</sup> <sup>74</sup>. This is conceptually similar and often easier to implement than actual 2D grid counting. But sticking to the box metaphor: for each  $\epsilon$ , one can step through time from 1 to  $N$  in increments of  $\epsilon$ , and for each time bin find the min and max amplitude of the signal in that bin. The set of amplitude bins between min and max are “filled” by the curve. Count those. However, this is essentially computing an upper bound; a more exact might require subdividing amplitude finely.
5. Alternatively, treat each line segment between sample  $i$  and  $i+1$  and mark all grid boxes it passes. There are known line drawing algorithms (like Bresenham’s) that could be adapted to see which grid cells a line goes through.

6. For coarse boxes, many points fall in one box; for fine boxes, the curve might pass through many.

However implemented, you obtain  $N(\epsilon)$ , the number of boxes of side  $\epsilon$  needed to cover the curve <sup>75</sup> <sup>76</sup>. For example, at the largest scale  $\epsilon = N$  (one big box), obviously  $N(\epsilon) = 1$  since the whole curve fits in one box. At the finest scale  $\epsilon = 1$  (boxes of 1 sample by 1 amplitude unit),  $N(1)$  would equal the number of distinct  $(i, x[i])$  pixel cells the line occupies – essentially the length in pixels (which is close to the total number of samples plus those in vertical moves). For fractal dimension, we're interested in how  $N(\epsilon)$  scales as  $\epsilon$  changes.

1. **Estimate the fractal dimension:** For a fractal-like curve,  $N(\epsilon)$  vs  $\epsilon$  often follows a power-law:

$$N(\epsilon) \sim \epsilon^{-D},$$

meaning  $N(\epsilon)$  is approximately proportional to  $\frac{1}{\epsilon^D}$  <sup>67</sup>. Taking logs,

$$\log N(\epsilon) \approx -D \log \epsilon + \text{const}.$$

Therefore, plot  $\log_{10} N(\epsilon)$  against  $\log_{10}(1/\epsilon)$  (or  $\log_{10} \epsilon$ ) and perform a linear regression on the linear portion of the curve. The **slope** of that line gives an estimate of the fractal dimension  $D$  <sup>71</sup> <sup>76</sup>. In practice, you'll have a discrete set of  $\epsilon$  values (like powers of 2) – do a least-squares fit of  $\log_{10} N$  vs.  $\log_{10} \epsilon$ . The resulting  $D$  should lie between 1 and 2 for a waveform curve. A perfectly smooth non-fractal curve (e.g. a straight line or a simple differentiable function) tends towards  $D=1$ . A highly erratic curve might approach  $D$  closer to 2 (but cannot reach 2 unless it fills an area). **Digital audio signals** are not truly fractal at arbitrarily fine scales (they have finite resolution and bandwidth), so typically the plot will be linear over some intermediate scales and then flatten out at the finest scales (where noise or sampling effects dominate). One chooses the linear region to fit. For instance, Sarkar & Chaudhuri (1994) suggested methods to optimize scale ranges for accuracy <sup>77</sup> <sup>78</sup>.

In the fractal speech processing literature, box-counting has been successfully used for **texture segmentation** of signals by looking at local FD <sup>33</sup> <sup>79</sup>. It's reported to work reliably for FD in range ~1 to 1.5 for 1D signals <sup>80</sup> – beyond that, estimates become less accurate due to finite data issues (e.g. extremely jagged signals saturate the method near 1.5) <sup>80</sup> <sup>81</sup>.

**Preprocessing:** As hinted, scaling and normalization are crucial: - **Aspect ratio normalization:** Because we treat time and amplitude in the same units for “boxes,” we must ensure a meaningful aspect ratio. If not normalized, imagine a very slow-varying but large-amplitude signal – time axis might span far more than amplitude or vice versa, which could bias the count. A safe route is to normalize time length = 1 (or  $N$  in samples if using sample units) and amplitude range = 1 (by scaling amplitude values into  $[0,1]$ ). This way, a “square” box has equal length in time and amplitude dimensions (in the normalized space). The FD result will then be invariant to original durations or amplitude magnitudes, which is desirable. In practice, if using an  $N \times N$  grid as earlier, we effectively did this by stretching amplitude to an  $N$ -scaled range. - **Detrending:** If the audio has a strong trend (e.g. slow fade in or out), that adds length to the curve but is not really “fractal” structure. Removing global trends or mean can help. However, in speech, trends are usually not an issue (no DC drift typically). - **Filtering:** Box-counting is sensitive to every little jaggedness (similar to Higuchi). If there is high-frequency noise, at the finest box scales the curve will occupy many boxes. If that noise is not what we consider part of the fractal nature but just extraneous noise, one might filter it out. In fact, *Al-Akaidi (2004)* mentions pre-filtering speech with a  $1/k$  filter to get fractal dimension in the correct range <sup>82</sup> <sup>83</sup>. This suggests that raw speech



might have a dimension outside expected due to certain spectral characteristics, and a filter was used to shape the spectrum (the filter  $1/k$  presumably flattens a  $1/f$  type spectrum). This is a bit technical, but the gist is: you may apply a frequency weighting to better reveal fractal self-similarity. For classification, though, one might not need that; it was more for segmentation tasks in that book. - **Silence:** A long silence (flat zero amplitude) will contribute almost no new occupied boxes as scale changes (basically a line of zero amplitude). It will bring the FD closer to 1. If analyzing an entire file with silence, you might get a lower dimension primarily reflecting the silent portion. So again, it's wise to exclude silence or treat it separately. Perhaps treat each continuous region of speech separately for FD and then combine. Or ensure your normalization of amplitude doesn't get skewed by including zeros (if you scale amplitude, exclude silence from the range). - **Data length and power-of-2:** Many implementations require  $N$  to be a power of 2 or will pad/truncate otherwise. Padding with zeros (silence) would lower FD as discussed. If needed, better pad with a low-amplitude noise or something neutral, but ideally just choose an  $N$  that covers the main content (or cut to nearest power of 2 around the content). - **Grid alignment:** Box-counting results can vary slightly depending on how the grid is aligned with the curve. Some use an "irregular grid" approach: shift the grid or average over a few alignments <sup>84</sup>. But for simplicity, a fixed alignment (say starting at the signal min and first sample exactly on a grid line) is fine, given we're not after super precise math but a feature for classification.

**Global vs. local:** Box-counting FD is typically a global measure because it needs a range of scales. However, it can be applied locally in a sliding window manner: compute the FD over each window by performing the above procedure on that segment of the waveform. This is computationally heavier than Katz or Petrosian for a moving window, but it was done in the literature for **fractal dimension segmentation (FDS)** of speech <sup>33 85</sup>. They found it effective in detecting changes in texture (e.g., vowels vs. consonants have different FD). For deepfake detection, one could similarly examine if certain phoneme classes show more discrepancy in FD between real and fake. For example, maybe the fractal dimension of an /s/ sound (fricative) is lower in a synthesized speech because it's less noisy than a real /s/. A local FD analysis could uncover that. If so, a classifier could exploit not just an overall FD but how FD varies across phonemes. That said, this is quite in-depth; a simpler approach is to compute one FD for the entire utterance (or for voiced vs unvoiced portions separately) and use those as features.

**Relevance to deepfake detection:** Box-counting is a direct measure of how much of the area the waveform's graph covers at different scales. If a deepfake speech waveform is **less complex**, it might "hug" a smoother path, occupying fewer boxes at fine scales compared to a real waveform. This would result in a lower slope (closer to 1). Conversely, if the deepfake contains weird high-frequency components (like slight ringing or tremolo), it could occupy more tiny boxes, potentially giving a higher dimension. The typical expectation might be that **deepfake speech is smoother (less complex)** due to model smoothing, yielding a slightly lower FD than natural speech, which contains physiological noise and richness. For example, real human speech has micro-level amplitude perturbations (due to muscle vibrations, turbulent airflow etc.) that a TTS might not fully replicate. These perturbations cause the waveform to deviate and fill more of the plane (hence higher Minkowski dimension). In contrast, synthesized audio might have a more regular waveform (except for possibly some periodic artifacts), meaning parts of the curve might be nearly repetitive or flat in small segments, thus not increasing the count of occupied boxes as rapidly at small scales.

Box-counting FD also implicitly captures both *temporal* and *amplitude* variation simultaneously, which is an advantage. Other features often separate time and frequency domains, but fractal dimension merges them into a single complexity score. If deepfake vs real differ in both domains, FD can reflect that. For instance, maybe deepfake has unnatural amplitude plateau or clipping (affecting amplitude distribution) and unnatural transitions (affecting time structure), and FD could pick up the combination.

One challenge is that box-counting FD can be a bit **unstable on finite data** and sensitive to parameter settings. But if implemented consistently, you can measure across a dataset and see if there's a systematic difference. If there is, that's a valuable feature. If not, the method might not add much beyond, say, what Higuchi's FD gave (Higuchi and box-counting often correlate since both measure curve length scaling). In comparisons on biomedical signals, box-counting was not included in the Esteller (2001) study, but others have found it generally reliable but slower.

**Implementation in pipeline:** - Box-counting is computationally more expensive than Katz or Petrosian. If we have  $M$  scales and the signal length  $N$ , a naive approach might be  $O(M \times N)$  or worse if scanning amplitude bins too. But since we often use dyadic scales,  $M = \log_2 N$ . For  $N=65536$  (approx 4 seconds at 16kHz),  $M=16$ . So maybe 16 iterations of scanning through data – not bad. The heavy part is counting boxes intersection which might involve nested loops or matrix operations. - One can optimize by using binary matrices: Represent the waveform on a high-res grid (e.g. an image array of size  $N \times N$  with 1's where the waveform passes). Then  $N(\epsilon)$  can be obtained by block-summing that matrix into coarser grids. This effectively leverages image processing – e.g., you can take the fine grid image and downsample it (via max or logical OR) to simulate larger boxes. Each downsample by 2 in each dimension corresponds to doubling  $\epsilon$ . By doing this progressively, you can count boxes by checking how many downsampled pixels are  $>0$ . This could be done with convolution or reshaping in numpy. However, constructing an  $N \times N$  image for large  $N$  might be memory heavy (e.g.  $65536 \times 65536$  is huge, not feasible). So usually one wouldn't go that fine; you might limit to a smaller representational grid or use line-scan logic. Perhaps using a moderate resolution like  $1024 \times 1024$  grid to approximate the FD of the signal is enough, since beyond that the sampling might be overkill. - There are known libraries or code for fractal dimension via box counting (commonly used in image analysis). Adapting one for 1D curves is possible. If needed, one could also implement the **divider method** (walking-divider) as an alternative: measure length with different step lengths and do log-log fit. The divider method is conceptually similar and might be easier to implement by code: you “walk” along the signal in steps of size  $\epsilon$  (horizontal) and sum vertical plus horizontal distances, which is similar to Higuchi but not averaging over start points. Many image fractal dimension codes actually do something equivalent behind the scenes. - In a batch pipeline, computing one box-count FD per utterance is definitely feasible for typical dataset sizes. If integrating with Python, one might rely on external code (maybe writing a C extension for speed if needed). But given we can choose scale steps not too fine, it's manageable. - **Parameters to tune:** The range of scales to use for the linear fit – exclude the extremes where the log-log might curve. Often the very largest scale and very smallest scale are not in the linear region. For example, largest scale  $\epsilon = N$  always gives  $N(\epsilon)=1$  which can be an outlier on log-log plot. The smallest scale  $\epsilon = 1$  might also be off if the signal isn't fractal beyond sampling limit. So one might fit from  $\epsilon = N/2$  down to  $\epsilon = \text{some small value}$ , but not all the way to 1. Alternatively, include all and trust the linear regression if it fits well (the correlation coefficient of the line fit can be checked to gauge fractality). - In terms of **spoof detection performance:** If box-counting FD shows significant differences between classes, it could even be used as a simple detector itself. But more likely it's one feature among many. There was research in image deepfake detection that used fractal geometry of residual noise patterns <sup>86 87</sup> – analogous ideas could be tried in audio by analyzing fractal properties of residual signals (audio minus a smoothed version). While the user's focus is waveform FD, note that one could also compute FD on spectrogram images of audio. In fact, *if allowed*, fractal dimension of a spectrogram could highlight texture differences (like local fractal dimension of the spectro-temporal pattern might differ for synthetic vs real). That, however, is beyond the direct waveform methods asked.

In conclusion, **Box-counting FD** applied to audio waveforms gives a more geometric perspective on complexity. It's less commonly used in audio than the time-domain methods above due to complexity, but it encapsulates similar information. If implemented properly in a pipeline, it yields an additional feature which, combined with others, can strengthen deepfake detection. It might help identify cases

where a fake's waveform is either too smooth or has unnatural complexity that stands out when measured across scales. Since modern deepfake detectors tend to use many features, including fractal ones has shown promise <sup>37</sup>, and box-counting is part of that arsenal.

---

**Summary & Best-Practice Integration:** In a state-of-the-art *audio deepfake detection pipeline*, one would extract a range of features including traditional spectral coefficients and novel features like fractal dimensions <sup>19</sup> <sup>37</sup>. The fractal dimension methods described – **MF DFA**, **Higuchi's FD**, **Katz's FD**, **Petrosian's FD**, and **Box-Counting FD** – each provide a different lens on signal complexity:

- **MF DFA** yields a **multifractal spectrum** capturing how variability at small vs. large fluctuations differ. It's powerful for characterizing signals with non-uniform fractality and is robust to trends, at the cost of higher computation. For deepfakes, one might use a few summary metrics from the spectrum (like Hurst exponent and spectrum width) as features.
- **Higuchi's FD** provides an accurate **monofractal dimension** estimate via a multi-scale length approach, good for capturing overall roughness. It's recommended for a reliable single FD value of the waveform <sup>24</sup>.
- **Katz's FD** gives a quick **global complexity index** using total length vs. diameter. It's unit-normalized and easy to compute, possibly amplifying certain differences making it useful for classification <sup>36</sup>.
- **Petrosian's FD** is an ultra-fast **oscillation count** based metric. Its value is nearly 1 for most signals, but slight differences (on the order of 0.01–0.05) could correlate with real vs fake if one class systematically has more/fewer zero-crossings. It essentially encodes something similar to average zero-crossing rate, which can be a telling feature.
- **Box-Counting FD** is a more **classical fractal dimension** that accounts for scaling in both time and amplitude. It might capture aspects that the others (which mostly consider vertical excursions) miss, since it considers the 2D occupancy of the waveform. It's heavier to compute but can be done offline in batch.

When using these in a Python pipeline, one should: - **Standardize sampling** (e.g., 16 kHz mono) and possibly apply a consistent pre-processing (like DC removal, optional filtering) to all audio. - **Denoise carefully if needed:** If the dataset contains varying background noises that are not intrinsic to the speech, one might apply a noise reduction to isolate the speech signal. However, if those noises are part of the challenge (e.g., a spoof might have less microphone noise), then keep them. - **Compute features frame-wise or utterance-wise:** For a straightforward approach, compute each FD for the entire utterance (or the voiced portion of it) to get one value per file. This yields a feature vector of, say, 5 dimensions (one per method). These can feed a classifier or be used in an analysis. Alternatively, compute them on a rolling window (e.g., 1-second windows with overlap) and aggregate statistics (mean, variance) – this gives a richer representation at the cost of complexity. - **Combine with other features:** Fractal features can augment a feature-based classifier (like an SVM or random forest) or can be appended to neural network embeddings (for example, one could concatenate fractal features to x-vector or spectrogram embeddings before a decision layer). The survey by Yi *et al.* (2023) emphasizes combining various features for robust detection <sup>88</sup> <sup>89</sup>. - **Batch processing:** Use vectorized operations as much as possible and process files in parallel if computing MFCCs, fractal features, etc. The fractal features above are lightweight except MF DFA (which might be the slowest due to polynomial fitting in many windows). Still, with modern hardware, computing an MF DFA on a few thousand-sample signal is quick (especially if just doing, say, DFA or a fixed small set of  $q$  values). If performance is an issue, one could reduce the number of scales or  $q$ -orders in MF DFA, as full multifractal spectrum might be overkill if a single exponent suffices.

To illustrate the utility: in a recent approach for cough detection (analogous scenario), four different fractal dimension methods were computed and arranged as features, and using them in a transformer model yielded detection accuracies around 97–98% <sup>20</sup> . This underscores that fractal-based descriptors carry discriminative information that deep learning models can exploit when raw waveforms or spectra alone aren't enough.

In conclusion, by **integrating fractal dimension computations** into the audio analysis pipeline – ensuring proper preprocessing (noise control, segmentation) and choosing the right method for global vs local analysis – one can extract powerful features that help distinguish **genuine speech's natural complexity** from the possibly telltale simplicity or irregularities of **spoofed (deepfake) speech**. These methods can be batch-processed in Python using numerical libraries and combined with conventional features to build a state-of-the-art classifier that is more resilient to the ever-improving quality of audio deepfakes.

### Sources:

- Esteller et al., “A Comparison of Waveform Fractal Dimension Algorithms,” IEEE TCSI 2001 – algorithms (Higuchi, Katz, Petrosian) and their evaluation <sup>25</sup> <sup>24</sup> <sup>36</sup> .
- Ihlen, “Introduction to multifractal detrended fluctuation analysis...,” Front. Phys. 2012 – tutorial on MFDFA steps <sup>6</sup> <sup>90</sup> .
- Kantelhardt et al., “Multifractal DFA of nonstationary time series,” Physica A 2002 – formal definition of MFDFA procedure <sup>8</sup> <sup>14</sup> .
- Vallat, *Entropy* library docs – formulas for Katz and Petrosian FD <sup>45</sup> <sup>59</sup> .
- Al-Akaidi, *Fractal Speech Processing*, 2004 – discussion of box-counting and other FD methods for speech <sup>75</sup> <sup>72</sup> .
- Yi et al., “Audio Deepfake Detection: A Survey,” arXiv 2023 – pipeline vs end-to-end detection frameworks <sup>19</sup> .
- Zaman et al., “Transformers and Audio Detection Tasks: An Overview,” DSP 2025 – example of using multiple FD features for audio classification <sup>37</sup> .
- Petrosian, *Proc. IEEE CBMS 1995* – original Petrosian FD (cited via Vallat and Esteller) <sup>91</sup> <sup>92</sup> .
- Others as cited above in context.

---

<sup>1</sup> <sup>2</sup> <sup>5</sup> Introduction\_to\_MFDFA.pdf

file:///file-3jziEiDxBxcy8tD4fkurxE

<sup>3</sup> <sup>4</sup> <sup>6</sup> <sup>7</sup> <sup>8</sup> <sup>9</sup> <sup>10</sup> <sup>11</sup> <sup>12</sup> <sup>13</sup> <sup>14</sup> <sup>15</sup> <sup>16</sup> <sup>87</sup> <sup>90</sup> Kantelhardt\_2002\_MFDFA (1).pdf

file:///file-VYm53YAH4DiGEdeR5LnTN

<sup>17</sup> <sup>18</sup> <sup>22</sup> <sup>23</sup> <sup>24</sup> <sup>25</sup> <sup>26</sup> <sup>27</sup> <sup>28</sup> <sup>29</sup> <sup>30</sup> <sup>31</sup> <sup>32</sup> <sup>35</sup> <sup>36</sup> <sup>38</sup> <sup>39</sup> <sup>40</sup> <sup>41</sup> <sup>42</sup> <sup>43</sup> <sup>44</sup> <sup>47</sup> <sup>49</sup> <sup>50</sup> <sup>51</sup> <sup>52</sup> <sup>53</sup> <sup>54</sup>  
<sup>55</sup> <sup>56</sup> <sup>57</sup> <sup>61</sup> <sup>62</sup> <sup>63</sup> <sup>64</sup> <sup>65</sup> <sup>91</sup> Fractal\_Dimension\_Algorithms.pdf

file:///file-CAdYV7mqP4dTwhCPQu6LHZ

<sup>19</sup> <sup>21</sup> <sup>88</sup> <sup>89</sup> 2308.14970v1.pdf

file:///file-343gRugqAwt2KEE8gj6kYG

<sup>20</sup> <sup>37</sup> (PDF) Transformers and Audio Detection Tasks: An Overview

[https://www.researchgate.net/publication/387395372\\_Transformers\\_and\\_Audio\\_Detection\\_Tasks\\_An\\_Overview](https://www.researchgate.net/publication/387395372_Transformers_and_Audio_Detection_Tasks_An_Overview)

<sup>33</sup> <sup>34</sup> <sup>68</sup> <sup>70</sup> <sup>71</sup> <sup>72</sup> <sup>73</sup> <sup>74</sup> <sup>75</sup> <sup>76</sup> <sup>77</sup> <sup>78</sup> <sup>79</sup> <sup>80</sup> <sup>81</sup> <sup>82</sup> <sup>83</sup> <sup>84</sup> <sup>85</sup> Fractal Speech Processing -- Al-Akaidi, Marwan, 1959- -- Cambridge, UK ; New York, 2004 -- Cambridge ; New York\_ Cambridge University Press -- 9780511192234 -- f8f975c489f5abf2b9ea7bd257c7feb0 -- Anna's Archive.pdf

file:///file-NVZuUZZdM9vyfGw2kC13EG

45 46 **entropy.katz\_fd — entropy 0.1.3 documentation**

[https://raphaelvallat.com/entropy/build/html/generated/entropy.katz\\_fd.html](https://raphaelvallat.com/entropy/build/html/generated/entropy.katz_fd.html)

48 **Fractals and the analysis of waveforms - ScienceDirect.com**

<https://www.sciencedirect.com/science/article/pii/0010482588900418>

58 59 66 92 **entropy.petrosian\_fd — entropy 0.1.3 documentation**

[https://raphaelvallat.com/entropy/build/html/generated/entropy.petrosian\\_fd.html](https://raphaelvallat.com/entropy/build/html/generated/entropy.petrosian_fd.html)

60 **(PDF) Fractal dimension characterizes seizure onset in epileptic ...**

[https://www.academia.edu/20668768/Fractal\\_dimension\\_characterizes\\_seizure\\_onset\\_in\\_epileptic\\_patients](https://www.academia.edu/20668768/Fractal_dimension_characterizes_seizure_onset_in_epileptic_patients)

67 69 **Fractal Dimension.pdf**

<file:///file-1jWxdk5bFTbb31vMeFL5Sm>

86 **Forensics Analysis of Residual Noise Texture in digital Images for Detection of Deepfake (2).pdf**

<file:///file-2pak63DadJKJPLG67GgJP7>